004.8

ДОЛГОВЕЧНАЯ ТИПИЗИРОВАННАЯ ПАМЯТЬ АВТОНОМНОГО АГЕНТА НА ГРАФЕ ЗНАНИЙ OSTIS

Ю. В. Ступчик, О. В. Ковалевская
Государственное учреждение образования «Дубненская средняя школа имени Андрея Степановича Данилова», Дубно, Беларусь
youlainvr@gmail.com

Введение

Автономные агенты, которые сами вызывают инструменты: читают файлы, правят код, запускают проверки, уже выполняют длительную работу без человека. Слабое место у них общее: память. История работы живёт либо в окне контекста языковой модели, либо в текстовом журнале, либо в таблице реляционной базы, вынесенной рядом с системой. У всех трёх способов один изъян: память не является частью самих знаний системы. Тип записи, порядок событий и происхождение утверждения восстанавливаются разбором формата строки, а не структурой данных. Стоит процессу завершиться аварийно или смениться машине, и ответить на вопрос, что именно агент помнил в момент сбоя, становится нечем.

Работа посвящена памяти агента в открытой семантической технологии проектирования интеллектуальных систем OSTIS [1, 2, 3], где база знаний является самой программой: элементы знаний, связи между ними и обрабатывающие их агенты живут в одном семантическом графе. Мы отвечаем на три вопроса. Как записывать опыт агента так, чтобы структура несла тип, порядок и происхождение записи? Как пережить аварийное завершение процесса и машины, не потеряв уже принятые записи? И как измерить границу потери, а не заявить её?

Вклад работы состоит из четырёх частей: типизированный протокол публикации ревизий, в котором цепочкой версий владеет сам граф, а не приложение над ним; ограниченные чтения, идущие по цепочке публикаций, а не перечисляющие класс целиком; механизм долговечности из явного сохранения, привязанного снимка, шлюза долговечности и барьера намерения; и правило, запрещающее слепой повтор операции с неизвестным исходом.

Память агентов активно исследуется как отдельный предмет: существуют публичные наборы задач для долговременной памяти ассистента [4], а накопленная история рассматривается и как источник стратегий, заменяющий повторный запуск среды [5]. В работах этого направления проверка утверждений опирается на символьную модель, которая позволяет отклонять неверные планы дешёво [6]. Наша работа добавляет к этому слою то, что обычно остаётся за кадром: поведение памяти при аварийном завершении и измеримую границу потери записей.

1. Требования к памяти

Мы формулировали требования до реализации, и каждое из них проверяемо.

R1. Типизация. Каждая запись принадлежит объявленному классу онтологии; тип проверяется и при записи, и при чтении, а несовпадение типов сообщается, а не исправляется молча.

R2. Порядок и происхождение. Последовательность записей и связь записи с её источником восстанавливаются по структуре графа, без разбора текста и без доверия к метке времени.

R3. Отсутствие теневого хранилища. Память не дублируется в SQLite или JSON. Вторичное хранилище расходится с основным при первом же сбое между двумя записями, и тогда непонятно, какое из двух считается истиной. По этой причине резервный путь записи в работе отсутствует: при недоступности графа запись не выполняется.

R4. Детерминированное восстановление. Повторный запуск после сбоя приводит память в согласованное состояние сам, без ручного вмешательства и без угадывания, была ли начатая операция выполнена.

R5. Граница потери. При аварийном завершении теряются только записи новее последней контрольной точки, и эта граница измеряется на стенде. Проектное требование: не более 300 секунд.

R6. Запрет слепого повтора. Если внешний эффект был начат, а его исход неизвестен, повтор не выполняется. Сначала проверка состояния, и только потом решение.

2. Онтология опыта и типы записей

Опыт хранится элементами графа. Онтология объявляет классы записей: сообщение разговора, задача, запуск, действие, решение, заметка, опыт, эксперимент, выпуск. Отдельные перечисления задают происхождение записи, её состояние и исход эффекта. Связи nrel_oneiro_previous и nrel_oneiro_previous_revision фиксируют предыдущую запись потока и предыдущую ревизию записи, а связь nrel_oneiro_publication связывает опубликованную запись с головой потока.

Расширение онтологии выполнено аддитивно и с проверкой типов: миграция читает фактические sc-типы существующих элементов, создаёт недопустимые ранее связи как не-ролевые отношения, а обнаруженные несовпадения типов сообщает списком, не перезаписывая найденное. Такая осторожность вызвана практикой: молчаливая замена типа делает граф синтаксически целым и семантически ложным.

Запись и чтение выполняют нативные агенты на C++, по одному агенту на операцию, а подготовку полей, которые нужны повтору, выполняет мост на Python. Прикладной код на Python общается с графом только через этот уровень,

поэтому онтология объявлена в одном месте, а не размазана по вызывающему коду.

Часть констант в нативных агентах пришлось исправлять по ходу работы: успех, отказ и признак отсутствия временной метки были объявлены как классы, тогда как онтология объявляет их узлами. Пока тип не совпадал, агент формально отрабатывал и не находил ничего, и ошибка выглядела как пустая память, а не как ошибка типа.

3. Протокол публикации ревизий

Запись одной заметки в графе это не одна операция, а несколько: создать узел, привязать содержимое, объявить тип, связать с предыдущей записью, сдвинуть голову потока. Между любыми двумя из них процесс может умереть, поэтому вопрос не в том, как записать быстро, а в том, как сделать незавершённую запись обнаружимой.

Протокол устроен так. Головой потока владеет сам граф: признак головы и не более одной незавершённой публикации хранятся связями в графе, а не в переменной процесса. Каждая публикация получает неизменяемую ревизию, а у записи есть указатель на текущую ревизию. Операция записи несёт устойчивый идентификатор и дайджест содержимого, а ожидаемая ревизия передаётся вызывающим: расхождение ожидаемой и фактической ревизии это конфликт, который возвращается ошибкой, а не тихо переписывается.

Именованные связи публикуются в порядке «сначала содержимое, потом идентификатор». Если процесс погиб между двумя шагами, в графе остаётся связь с готовым содержимым и без объявленного имени, и восстановление доводит её до конца. Обратный порядок дал бы имя без содержимого, то есть запись, которая на вид существует и при чтении ничего не отдаёт.

Восстановление устроено идемпотентно. При открытии потока незавершённая публикация проверяется и завершается до того, как принимается новая работа; читать поток с незавершённой публикацией запрещено, чтобы наружу не уходило полусостояние. Повторное выполнение операции с тем же идентификатором принимается ровно один раз: если операция не начиналась, повтор её выполняет, если начиналась и завершена, повтор возвращает уже опубликованный результат. Такая проверка делалась на каждом испытанном рубеже записи: до записи, между шагами и после, и во всех случаях повторный запуск приводил память в согласованное состояние.

Отдельное правило касается операций с внешним эффектом. Намерение выполнить эффект записывается в граф до самого эффекта, и такая запись несёт признак, требующий проверки. Запись с неизвестным исходом не повторяется и не считается выполненной: она остаётся в состоянии, которое человек или агент обязан разрешить проверкой.

4. Ограниченные чтения

Чтение устроено как просмотр страницы цепочки публикаций, а не как перечисление класса. Обычный способ получить записи класса это обойти все его экземпляры и отфильтровать; стоимость такого обхода растёт с размером графа и не зависит от того, сколько записей нужно показать. Мы читаем от головы потока назад, по связям ревизий, и останавливаемся, набрав страницу.

Курсор страницы фиксирует окно вставки, а не адрес: это важно, потому что адреса узлов переиспользуются, и курсор по адресу после перезапуска указал бы в другое место. Поля страницы явно сообщают о неполноте: сколько записей пропущено, не усечён ли ответ, и какой курсор продолжает выборку. Содержимое одной записи ограничено сверху: 131 072 байта для связи, 1024 байта для курсора; превышение это ошибка, а не молчаливое усечение.

Проверка принадлежности записи классу выполнена с опорой на тот конец связи, у которого меньше связей: поиск идёт от экземпляра к классу по входящим связям, а не по всем экземплярам класса. Кэш именованных адресов ограничен 1024 записями, пересобираем и никогда не кэширует отсутствие или содержимое, потому что кэш отрицательного ответа превращает временную недоступность в устойчивую ложь.

5. Долговечность

Долговечность в работе это не свойство отдельной записи, а отношения между памятью и носителем, на котором она лежит. Механизм состоит из четырёх частей.

Явное сохранение. Граф сохраняется не «когда-нибудь», а явным действием: нативный агент вызывает поддержанный API сохранения контекста памяти, а мост ждёт наблюдаемого завершения действия, а не просто того, что данные оказались в оперативной памяти. Обратное прочтение из памяти не считается доказательством долговечности: оно доказывает лишь то, что запись видна тому же процессу.

Привязка снимка. Снимок делается не «по пути», а по проверенной привязке: контейнер, том и каталог, предельный размер, квота на все хранимые поколения и зарезервированный свободный объём диска. Перед сохранением привязка проверяет, что том действительно смонтирован, а опубликованный порт 127.0.0.1 принадлежит именно этому серверу графа. Без этой проверки можно снять снимок другого сервера и открыть шлюз долговечности по чужим данным.

Порядок снимка. Сначала сохранение, затем синхронизация файлов внутри контейнера, затем ограниченное чтение архива потоком, затем проверка архива (хеши файлов, типы, пути), затем синхронизация архива и манифеста на диск. Хранятся все удержанные поколения; при достижении квоты или запаса свободного места шлюз закрывается, а не удаляет предыдущее поколение молча.

Шлюз долговечности и барьер намерения. Шлюз считает, покрыта ли запрошенная последовательность записей уже сделанной контрольной точкой. Возраст считается от начала цикла, а не от его конца, чтобы длительность снимка не съедала запас. Если запись новее последней контрольной точки, вызов эффекта отклоняется с признаком, что намерение ещё не долговечно. Пока сохранение не покрыло последовательность, внешний эффект не выполняется. На стенде это проверено прямым опытом: барьер отказал действию новее последней контрольной точки, затем сохранение, синхронизация и архив покрыли его последовательность, и только после этого действие было разрешено. Внешний эффект при этом не выполнялся, а повтор записи с неизвестным исходом остался отклонён.

Наблюдение за долговечностью вынесено в супервизор с конечными сроками: интервал и предельная длительность каждого цикла заданы и ограничены сверху, а исчерпание сроков или сбой цикла закрывает шлюз, а не оставляет память «вероятно долговечной».

6. Реализация

Память встроена в продукт: HTTP-маршруты чтения записей и чтения одной записи работают через тот же протокол, а не через отдельную ветку кода для интерфейса. Клиент графа вынесен в отдельный процесс с ограниченным числом одновременных запросов, границей размера запроса и предельным сроком обработки, включающим запуск, отправку и приём. При зависании или отмене завершается только этот процесс; сервер графа не перезапускается, а незавершённая публикация восстанавливается при следующем обращении. Обрыв связи с сервером отличается от отсутствия данных и сообщается отдельной ошибкой.

Точка входа контейнера переписана так, чтобы исключить самую дорогую ошибку: она больше не очищает существующее хранилище даже при исторической строке подтверждения и завершается отказом, если хранилище отсутствует или недоступно. Ошибочное подтверждение не должно уметь стереть данные, которые никто не архивировал.

Проверка велась не только на продукте. Набор тестов памяти, графового адаптера, рабочего процесса и миграции проходит целиком (129 тестов после последнего изменения адресного кэша и квоты, 525 тестов на пересобранном графе с онтологией и нативными агентами), а статическая проверка типов проходит без замечаний на 17 модулях. Это не доказательство поведения, но это защита от возврата уже исправленных ошибок.

7. Результаты измерений

Измерения делались на стендовой конфигурации: контейнер с графом знаний, отдельный том для данных, ограниченный набор записей и принудительное завершение процесса. Каждое число ниже воспроизводимо скриптом и лежит в протоколе вместе со снимком графа.

Аварийное завершение и независимое восстановление. Стенд с 20 заметками и одним действием с неизвестным исходом (таблица 1).

Таблица 1. Восстановление после жёсткого завершения контейнера

Показатель	**Значение**
Подготовлено записей	20 заметок и 1 действие с неизвестным исходом
Длительность явного сохранения	0,039479 с
Длительность независимого снимка	0,336536 с
Размер снимка	12 929 024 байта
Жёсткое завершение	через 16,085290 с после начала контрольной точки
Пережили завершение и перезапуск	21 запись из 21
Проверено после перезапуска	типы, членство в классах, связи публикации и предыдущей ревизии
Независимое восстановление	на отдельном томе, без записей починки

Запись с неизвестным исходом пережила завершение и по-прежнему отказывает в слепом повторе: восстановление возвращает её в состояние, требующее проверки, а не в состояние «выполнено».

Периодическая граница потери. Отдельный опыт длился 140 секунд с автоматическими контрольными точками внутри (таблица 2). После принудительного завершения и перезапуска память восстановилась на последней контрольной точке.

Таблица 2. Граница потери при периодических контрольных точках (опыт 140 с)

Показатель	**Значение**
Номера контрольных точек в опыте	1, 2, 25
Длительность цикла контрольной точки	0,770282 / 0,879942 / 0,768908 с
Создано записей	28
Пережили завершение	24
Потеряно	4 записи новее последней контрольной точки
Возраст старейшей потерянной записи	20,501997 с

Измеренная граница потери в этом опыте составила около 20,5 секунды, то есть заметно меньше проектного требования в 300 секунд. Оговорка существенна: измерение сделано на малом графе и на одном сценарии завершения, поэтому оно подтверждает порядок величины, а не гарантию для любого графа.

Стоимость чтения страницы. Пять повторов чтения страницы при двух размерах потока (таблица 3). Оба замера выполнены на прогретом кэше адресов, поэтому сравнивать их между собой как ускорение нельзя.

Таблица 3. Чтение страницы записей из графа

Записей в потоке	**Повторов**	**Медиана**	**Максимум**	**Чтений связи**	**Обращений к серверу**
1 000	5	0,599577 с	0,607110 с	51	153
10 000	5	0,371774 с	0,372880 с	51	102

Существенно не время, а то, что число чтений связи не растёт вместе с потоком: страница читается по цепочке публикаций и остаётся ограниченной. В первом же замере на 10 000 записей пик по трассировке аллокаций составил 182 711 байт, а резидентная память процесса изменилась с 236 351 488 до 236 908 544 байт, то есть чтение страницы не тянет за собой весь поток.

8. Ограничения

Границы работы названы прямо, потому что именно они определяют, что можно переносить в практику.

Договор об единственном писателе обеспечен блокировкой в пределах одной машины. Это не распределённая блокировка и не утверждение о транзакционности: два независимых писателя в один поток протоколом не поддержаны.

Явное сохранение не является атомарным для всего хранилища. Оно требует, чтобы писатели графа были остановлены на время сохранения, и не заменяет собой ни репликацию, ни журнал предзаписи. Завершение процесса ровно в момент сохранения, потеря питания и завершение сервера извне отдельно не проверялись; проверено завершение между операциями записи.

Измерения проведены на малом графе: снимок занимает около 13 МБ. Граница потери на графе в сотни мегабайт не измерена, как и поведение при большом числе одновременных читателей, при переподключении клиента и при частично записанном ответе. Чтения измерены на прогретом кэше; холодный кэш и потоки порядка ста тысяч записей остаются незакрытыми пунктами.

Наконец, влияние вида памяти на качество работы агента в этой работе не измеряется: предметом были структура, восстановление и граница потери. Сравнение типизированного графа с плоским журналом на публичном наборе задач требует отдельного протокола с судьёй и повторами и в этот доклад не входит.

Заключение

Память автономного агента можно сделать частью его знаний, а не приложением рядом с ними. В работе это сделано на графе знаний OSTIS: типизированные записи опыта ложатся в граф, цепочкой ревизий и незавершённой публикацией владеет сам граф, чтение идёт по цепочке публикаций с ограниченной стоимостью независимо от размера потока, а долговечность обеспечена явным сохранением, проверенным снимком и шлюзом, который не разрешает внешний эффект до того, как покрыта его последовательность.

Измерения подтверждают выбранный порядок величин: 21 запись из 21 переживает жёсткое завершение контейнера и восстанавливается на отдельном томе без починки, а периодический опыт даёт границу потери около 20,5

секунды при проектном требовании 300 секунд. Запись с неизвестным исходом ни при одном из сценариев не была выполнена повторно.

Дальше работу определяют названные ограничения: измерение на графе реального размера, завершение процесса в момент сохранения, потеря питания, несколько независимых писателей, холодный кэш и потоки в сотни тысяч записей. Отдельное направление это сравнение типизированной памяти с плоским журналом по протоколу с судьёй и повторами, чтобы преимущество структуры было видно не только в порядке, происхождении и восстановлении, но и в качестве работы агента.

Список использованных источников

1. Голенков В. В., Гулякина Н. А. Открытая семантическая технология проектирования интеллектуальных систем (OSTIS) // Сборник научных трудов OSTIS. Минск, 2023. URL: https://proc.ostis.net/proc/Proceedings%20OSTIS-2023.pdf.
2. Голенков В. В. и др. Интеллектуальные компьютерные системы нового поколения // Доклады БГУИР. 2024. URL: https://doklady.bsuir.by/jour/article/view/3906.
3. Технология OSTIS: программная платформа (sc-machine, sc-web, py-sc-client). URL: https://github.com/ostis-ai.
4. Wu D. et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory // arXiv:2410.10813, 2024. URL: https://arxiv.org/abs/2410.10813.
5. Dream-RSI: Recursive Self-Improvement through Evolving Worlds // arXiv:2609.14858, 2026. URL: https://arxiv.org/abs/2609.14858.
6. Wang R., Hsu H.-L., Mehta S. и др. GAVEL: Graph World Models for Verified and Efficient Long-Horizon LLM Task Planning // arXiv:2609.19315, 2026. URL: https://arxiv.org/abs/2609.19315.